

RAG-Based Customer Support Assistant

Technical Documentation

Name: T. Bhargavi

Intern ID: IN226082802

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) combines information retrieval with language generation. Instead of relying only on the LLM's internal knowledge, the system retrieves relevant content from a knowledge base and uses it to generate accurate answers.

Why is it needed?

Traditional chatbots often produce generic or incorrect answers. RAG improves accuracy by grounding responses in real documents, reducing hallucinations.

Use Case Overview

This project implements a customer support assistant that answers user queries using a PDF knowledge base. It retrieves relevant sections and generates context-aware responses, with escalation to human agents when needed.

2. System Architecture Explanation

The system consists of two main flows:

1. Document Ingestion (Offline)

PDF → Chunking → Embeddings → ChromaDB

2. Query Processing (Online)

User → UI → LangGraph → Retriever → LLM → Routing Layer → Response/HITL

- The **Retriever** fetches relevant chunks from ChromaDB
- The **LLM** generates answers using retrieved context
- The **Routing Layer** decides whether to return the answer or escalate
- The **HITL system** handles complex queries

3. Design Decisions

- **Chunk Size (500–1000 tokens)**
Balances context completeness and retrieval efficiency
- **Embedding Strategy**
Dense vector embeddings enable semantic search instead of keyword matching
- **Vector Store (ChromaDB)**
Chosen for simplicity, local setup, and fast retrieval
- **Retrieval Approach**
Top-k similarity search to fetch most relevant chunks
- **Prompt Design**
The LLM is given retrieved context + user query to generate grounded responses

4. Workflow Explanation (LangGraph)

LangGraph is used to control the execution flow.

- **Processing Node**
 - Handles retrieval and response generation
- **Output Node**
 - Returns final response to the user

- **State Transitions**

The state carries:

- Query
- Retrieved context
- Generated response

Flow:

Input → Processing Node → Output Node

5. Conditional Logic

The system uses conditional routing to decide the next step.

- **Intent Detection**

Identifies whether the query can be answered using available data

- **Routing Decisions**

- If sufficient context → generate answer
- If insufficient context → escalate

- **Criteria**

- Similarity score threshold
- Presence of relevant chunks

6. HITL Implementation

Human-in-the-Loop (HITL) ensures reliability.

- **Role**

Handles complex or unclear queries

- **Process**

Query is forwarded to human agent → response returned to user

- **Benefits**

- Improves accuracy
- Handles edge cases

- **Limitations**

- Increases response time
- Requires human availability

7. Challenges & Trade-offs

- **Retrieval Accuracy vs Speed**

Higher accuracy may increase latency

- **Chunk Size vs Context Quality**

Larger chunks improve context but reduce precision

- **Cost vs Performance**

Better models give better results but increase cost

8. Testing Strategy

- **Approach**

Test with different types of queries:

- Simple factual queries
- Multi-step queries
- Out-of-context queries

- **Sample Queries**

- "What is the refund policy?"

- "How can I reset my password?"
- "Explain something not in document"

- **Expected Results**

- Correct answers for known queries
- HITL escalation for unknown queries

9. Future Enhancements

- Support multiple documents
- Add feedback loop for learning
- Implement conversational memory
- Deploy as a web application